S1 Text. Event decoding, valuation, label formulas and extraction queries.

This text gives what the main text summarizes: which logs were read, how each field was decoded and valued, how every score input, check and label was computed, and which properties of the data were not quantified. Constants are those of the analysis pipeline; the repository and the commits that hold them are named in the Data availability statement. All results of the main text belong to the revised analysis (amounts in US dollars over five tokens, scores from the twelve months before each freeze); where the registered analysis differs, the difference is stated.

1. Source, extraction and score windows

All records are rows of the public BigQuery table `bigquery-public-data.goog_blockchain_arbitrum_one_us.logs` (Arbitrum One, chain identifier 42161). Each row is one event log with its block timestamp, block number, log index, emitting contract, topics and data. One query (scan A) copied the logs from 1 October 2023 00:00:00 to 30 June 2026 23:59:59 UTC that are ERC-20 `Approval` or `Transfer` logs with exactly three topics, GMX V2 `PositionDecrease` logs, or Aave V3 pool events (which no analysis of this article uses) into a table of the authors' project, partitioned by month. A second query (scan B) did the same for 1 July to 30 September 2026; its script refuses to run unless the W1 registration and the configuration are committed with the status "registered", and it records their SHA-256 hashes in an extraction manifest. Every later query reads the project tables. Each query was first run as a dry run and checked against a budget of USD 50 for the whole study; the jobs of the study billed 6.76 TiB and cost USD 42.24.

Every score, check and degree baseline of the revised analysis uses the events of one score window, the twelve months before its freeze (both ends inclusive; each freeze is at 23:59:59 UTC of its date):

Freeze	Score window	Label window
T_{obs}, 30 June 2026 (same window and W1)	1 July 2025 to 30 June 2026	1 July to 30 September 2026
t_1, 31 March 2026 (W0)	1 April 2025 to 31 March 2026	1 April to 30 June 2026

Each freeze has its own cohort: the contracts of the plan's cohort (27,844 contracts with code that received non-zero approvals of any ERC-20 token from at least three distinct owners in scan A) that at least three distinct owners approved, with a non-zero `Approval` of one of the five tokens of Section 3, inside the freeze's score window. The cohort of T_{obs}, the matched cohort, holds 6,483 contracts and that of t_1 holds 6,343, of which 5,383 are also in the matched cohort. The graphs, checks and labels of a freeze use the events of the five tokens with a contract of its cohort at one end, inside its score window for the graphs and checks and inside its label window for the labels. In the registered analysis every score used every extracted event from 1 October 2023 up to its freeze, and the cohort was the plan's cohort.

2. Account type

We asked public Arbitrum One RPC endpoints for the code stored at each address (`eth_getCode`) at one pinned block. Non-empty code makes it a contract. Empty code, or code consisting of the EIP-7702 delegation designator (`0xef0100` followed by a 20-byte address), makes it an externally owned account (EOA). The candidate spenders of the analysis plan were read at block 513,216,506 on 9 October 2026; holdout spenders and GMX accounts that were not candidates were read by the same call at later blocks within three hours of that read. Each read is stored with its block number. The state read is that of October 2026: an address whose code was deployed only after it had received approvals is treated as a contract throughout, and a contract that destroyed itself before EIP-6780 took effect has no code and counts as an EOA.

3. Tokens and prices

Token rule. Three conditions admit a token. (i) In the Chainlink directory of Arbitrum One data feeds (cited in the main text), a standard market-price feed in US dollars or in ether exists whose market-risk rating is low or medium. (ii) The feed prices the token itself, an asset that the token's issuer wraps one to one (as WETH wraps ether), or, for a token minted by Arbitrum's canonical bridge, the Ethereum token it represents. (iii) The token appears among the extracted transfers or latest allowances. Of the 133 tokens that the directory maps, 64 are listed and 46 of these have a daily DefiLlama price. The analysis keeps the five priced listed tokens with the most rows in the score window of T_{obs} (ego transfers of the plan's cohort in the window plus positive latest allowances at T_{obs} set inside it), which hold 79.40% of the rows of all tokens there:

WETH	`0x82af49447d8a07e3bd95bd0d56f35241523fbab1`	18 decimals	feed ETH / USD
USDC	`0xaf88d065e77c8cc2239327c5edb3a432268e5831`	6 decimals	feed USDC / USD
USDT0	`0xfd086bc7cd5c481dcc9c85ebe478a1c0b69fcbb9`	6 decimals	feed USDT / USD
ARB	`0x912ce59144191c1204e64559fe8253a0e49e6548`	18 decimals	feed ARB / USD
WBTC	`0x2f2a2543b76a4166549f7aab2e75bef0aefc5b0f`	8 decimals	feed WBTC / USD

All five feeds are rated low market risk. Every other token, including bridged USDC.e, is left out, and its rows are counted (S1 Table and the published data description).

Prices. The price of a token on a UTC day is the point of DefiLlama's daily chart under the token's Arbitrum address that lies nearest to 00:00 UTC of the following day. That series begins in August 2024 for WETH and WBTC and in January 2025 for USDT0, before the first score window opens, so each of the 548 days from 1 April 2025 to 30 September 2026, which the score and label windows cover, is priced under the token's own address. A point more than twice or less than half of both neighbouring points, while those lie within a factor of two of each other, would be replaced by the previous day's price, and a day without a point would take the previous day's price; neither case occurs for the five tokens on these days. On 20 of these days per token the DefiLlama price was compared with the Chainlink answer in force at 00:00 UTC of the next day, read on the chain: the median absolute relative difference is 0.010% to 0.056% per token and the largest 0.24%, for WBTC.

Valuation. The dollar value of a transfer is $x = v\,10^{-q}p$, with v its amount in base units, q the token's decimals and p the token's price on the transfer's UTC day. Latest allowances are valued at the price on the day of the anchor (30 June 2026 for T_{obs}, 31 March 2026 for t_1). The 32-byte amount is read as a double-precision number; an unlimited allowance ($2^{256}-1$ base units) thereby has a dollar value far above any slope or cap below. In the registered analysis amounts stayed in raw base units of every ERC-20 token.

4. ERC-20 approvals and the allowance graph

Selection and fields. Topic 0 equals `0x8c5be1e5ebec7d5bd14f71427d1e84f3dd0314c0f7b2291e5b200ac8c7c3b925`, the keccak-256 hash of `Approval(address,address,uint256)`, and the log has exactly three topics, which leaves out ERC-721 logs (they index the token identifier as a fourth topic). Topic 1 is the owner and topic 2 the spender (the last 20 bytes of each 32-byte word); the emitting contract is the token; the data field is the allowance. An EIP-2612 `permit` emits the same event and is therefore included. An approval of zero is kept, because it sets the allowance in force to zero. A data field that cannot be read as a number gives zero.

Latest allowance. For each (token, owner, spender) triple the record with the largest (block number, log index) inside the score window, up to the anchor, is kept. If its value is zero the triple is removed. With $\Delta t^{\star}$ the age in days of the kept record at the anchor and $a^{\star}$ its dollar value, the EndorseRank weight of the pair is

$$w_A(u,v) = \sum_{\text{token}} \sigma(\Delta t^{\star})\, V_A(a^{\star}), \qquad \sigma(\Delta t) = \frac{1}{1+e^{0.01(\Delta t-180)}},$$

$$V_A(z) = \frac{2}{1+e^{-b_A z}} - 1, \qquad b_A = \frac{\ln 3}{100} \approx 0.0110.$$

In floating point V_A is set to one from $b_A z \geq 40$. Pairs whose weight is zero in double precision are dropped. The amount-weighted allowance layer sums $\min(a^\star, U)$ over tokens with $U = 19{,}000$ dollars. The in-approve degree of a contract is the number of owners with a positive latest allowance to it, and its in-approve value is the sum of $\min(a^\star, U)$ over its owners and tokens.

Constants. The constants are computed once, on the W0 score window and the ego events of the W0 cohort, and used at both freezes. $m_A = 100$ dollars is the median, at t_1 prices, of the finite positive latest allowances at t_1 set inside that window (99.97 dollars, rounded to two significant digits). U is the 99th percentile of the dollar values of the single ego transfers in that window (18,575.3 dollars, rounded to two significant digits). Neither uses an event after t_1, so no label window enters them.

Known properties not quantified in this study. (a) Many ERC-20 implementations also emit `Approval` from inside `transferFrom` when they reduce a finite allowance. For such tokens the kept record may be the remainder after the spender's last use, not a new authorization: its age then dates the last spend, an allowance spent in full ends at zero and drops out, and an authorization from before the score window that is used during a label window can be counted as a new approval pair. The share of `Approval` logs emitted in the same transaction as a `Transfer` by the same token with the same owner and spender would measure this; it was not computed. (b) Contracts that manage allowances of their own, such as Uniswap's Permit2, record those inner allowances in events with a different signature, which the extraction does not read. Owners approve such a manager through an ordinary ERC-20 approval, so it appears as one spender, and the contracts it authorizes in turn are not seen. (c) Only the approvals inside a freeze's score window are read for its graph: an allowance that is still in force at the freeze but was last approved before the window opened has no edge, and the graph records only what owners authorized during those twelve months.

5. ERC-20 transfers and the transfer graph

Topic 0 equals `0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef`, the keccak-256 hash of `Transfer(address,address,uint256)`, with exactly three topics. Topic 1 is the sender, topic 2 the recipient and the data field the amount. Zero-value transfers are dropped. Mint and burn logs (zero address at one end) are kept when a cohort contract is at the other end. A self-transfer stays in the events and becomes a self-loop in the AWP graph, but it is excluded from the transfer in-degree baseline and from the new-sender label. The AWP weight and restart weight are

$$w_T(u,v) = \sum_{e:\, u \to v} \sigma(\Delta t_e)\, V_T(x_e), \qquad X_u = \sum_{v \neq u} \max_{e:\, u \to v} \sigma(\Delta t_e),$$

$$V_T(z) = \frac{2}{1 + e^{-b_T z}} - 1, \qquad b_T = \frac{\ln 3}{170} \approx 0.0065,$$

with $m_T = 170$ dollars, the median dollar value of the ego transfers in the W0 score window (169.33 dollars, rounded to two significant digits). Only transfers inside the score window enter the weights, and Δt_e runs from 0 to 365 days. The amount-weighted transfer layer sums $x_e\, \sigma(\Delta t_e)$. A spender that moves tokens with `transferFrom` directly from an owner to a third address does not appear in the resulting `Transfer` event. GMX V2 settles collateral and profits in ERC-20 tokens, so payouts from GMX contracts can appear in the pre-freeze transfer graph; their share of edges was not computed.

6. GMX V2 position decreases

Selection. GMX V2 emits its position events through one EventEmitter contract, `0xc8ee91a54287db53897056e12d9819156d3822fb` on Arbitrum One. Scan A and scan B keep the logs of that contract whose topic 0 equals `0x137a44067c8961cd7e1d876f4754a5a3a75989b4552f1843fc69c3b372def160` (the `EventLog1` event) and whose topic 1 equals `0x07d51b51b408d7c62dcc47cc558da5ce6a6e0fd129a427ebce150f52b0e5171a`, the keccak-256 hash of the event

name `PositionDecrease`. Topic 2 is the account, so the accounts can be listed without decoding; the closes of the accounts that hold contract code are then decoded.

Decoding. Each log is decoded with the EventEmitter ABI. The account is the `account` entry of the address items (topic 2 as a fallback); the profit or loss is the `basePnlUsd` entry of the signed integer items, a 30-decimal fixed-point value in US dollars; the order type is the `orderType` entry of the unsigned integer items; `sizeDeltaUsd` and `isLong` are read but enter no label. A log without `basePnlUsd` counts as a decoding error; all 133,685 closes of contract accounts from October 2023 to June 2026 and all 6,805 from July to September 2026 decoded without error.

Close and liquidation. Every `PositionDecrease` log is one close, whether it reduces a position partly or closes it. A close is a liquidation if its order type equals 7, the value of `Liquidation` in GMX V2's order-type enumeration, or if the event's `msgSender` is the liquidation handler `0xaf157eb8e2398a8e1fc1da929974652b9ba9bc25`. `basePnlUsd` is used as recorded; fees that GMX reports in separate events are not subtracted.

7. Label formulas

Spenders. New approval pairs of spender v: the number of distinct owners u that emit a positive `Approval` of one of the five tokens for v inside the label window while the pair (u, v) held no positive latest allowance, set inside the score window, at the freeze. New transfer senders of v: the number of distinct addresses $u \neq v$ that send v a transfer of one of the five tokens inside the label window and sent it none between the start of the score window and the freeze. Both are counted on events with a contract of the freeze's cohort at one end, the same filter as the graphs.

Further spender counts. Two counts that the plan did not name were computed on the same spenders; the main text reports them without offering them as evidence. Revocations of v: the number of owners u whose pair (u, v) held a positive latest allowance at the freeze and who emit an `Approval` of zero for v inside the label window. Drained owners of v: the number of owners u that give v a positive approval inside the window after which, within six hours, an outgoing transfer from u in the same token moves at least half the approved amount; for an approval of 10^{30} base units or more, any outgoing transfer of that token from u within one hour counts. Both use the five tokens only.

Traders. For a contract i let n_i be its closes in the label window, L_i the liquidated ones, W_i the closes that were not liquidations and had `basePnlUsd` > 0, and B_i the closes that were not liquidations and had `basePnlUsd` < 0. Labels are defined only when $n_i \geq 3$:

$$\begin{aligned} \text{liquidation-free close rate} &= 1 - L_i/n_i, & \text{no-liquidation flag} &= \mathbf{1}[L_i = 0], \\ \text{profitable closes} &= W_i, & \text{realized gain} &= \textstyle\sum_{\text{closes}} \max(\texttt{basePnlUsd}, 0), \\ \text{profitable share} &= W_i/n_i, & \text{non-loss share} &= 1 - B_i/n_i. \end{aligned}$$

Two consequences follow. Because partial decreases count as closes, a trader who reduces positions in many steps has more closes per liquidation and a higher liquidation-free close rate than one who closes in a single step; a position-level rate was not computed. Because a liquidated close has neither W nor B status, it counts against the profitable share but in favor of the non-loss share; the non-loss share therefore rises with liquidations by construction.

Stratified coefficient. For the stratified rows the traders are split by their number of label-window closes into four strata (3–4, 5–9, 10–24, and 25 or more). τ_b is computed in every stratum that holds at least five traders, and these coefficients are averaged with weights proportional to the number of trader pairs in each stratum.

8. Same-window checks

For a cohort contract w, with $\mathcal{T}_{\text{in}}(w)$ the transfers it received in the score window of T_{obs} (self-transfers included) and u_e the sender of transfer e: in-degree $= |\{u_e\}|$; in-value $= \sum_e x_e$ in dollars; in-approve degree and in-approve value as in Section 4. The Sybil-stability family uses the ratio of distinct inbound counterparties to inbound transfers from other addresses; the transfer tenure, the span in days between the first and the last transfer attributed to w (a transfer between two cohort contracts is attributed to its sender); and the number of active months, the larger of the months in which w received a transfer from another address and the months of the transfers attributed to it. Inside the score window the tenure is at most 365 days and the active months at most 12. A contract absent from a check's source is given zero. Each family mean is the plain average of the Kendall coefficients of its checks.

9. Query patterns

The listings are abridged; the full files are in `sql/` of the experiment folder. Names in double underscores are replaced before a query runs, and names that start with `@` are query parameters.

```
-- sql/01_materialize_logs.sql (scan A and scan B, one window each; Aave branch omitted)
CREATE TABLE `__DEST__` PARTITION BY TIMESTAMP_TRUNC(block_timestamp, MONTH)
CLUSTER BY kind AS
WITH src AS (
  SELECT block_timestamp, block_number, log_index,
         LOWER(address) AS address, topics, data
  FROM `bigquery-public-data.goog_blockchain_arbitrum_one_us.logs`
  WHERE block_timestamp >= TIMESTAMP(@start_ts)
    AND block_timestamp <  TIMESTAMP(@end_ts)),
tagged AS (
  SELECT *, CASE
    WHEN ARRAY_LENGTH(topics) = 3 AND topics[OFFSET(0)] = '0x8c5be1e5...' THEN 'A'
    WHEN ARRAY_LENGTH(topics) = 3 AND topics[OFFSET(0)] = '0xddf252ad...' THEN 'T'
    WHEN address = '0xc8ee91a54287db53897056e12d9819156d3822fb'
     AND topics[OFFSET(0)] = '0x137a4406...'
     AND topics[OFFSET(1)] = '0x07d51b51...' THEN 'G'
    END AS kind
  FROM src)
SELECT block_timestamp, block_number, log_index, kind,
  FROM_HEX(SUBSTR(address, 3)) AS emitter,
  IF(kind IN ('A','T'), FROM_HEX(SUBSTR(topics[OFFSET(1)], 27)), NULL) AS a,
  IF(kind IN ('A','T'), FROM_HEX(SUBSTR(topics[OFFSET(2)], 27)), NULL) AS b,
  IF(kind IN ('A','T'), SAFE.FROM_HEX(SUBSTR(data, 3)), NULL) AS val,
  IF(kind = 'G', topics, NULL) AS raw_topics,
  IF(kind = 'G', data, NULL) AS raw_data
FROM tagged WHERE kind IS NOT NULL;

-- sql/03_ego_events.sql: approvals (and, with kind = 'T' and zero amounts
-- dropped, transfers) with a cohort contract at one end
SELECT l.block_timestamp, l.block_number, l.log_index,
       l.emitter AS token_address, l.a AS owner, l.b AS spender,
       u256(l.val) AS value
FROM `contract_rep.__SRC__` AS l
WHERE l.kind = 'A'
  AND (l.a IN (SELECT address FROM `contract_rep.cohort`)
    OR l.b IN (SELECT address FROM `contract_rep.cohort`));

-- sql/12_usd_ego.sql: ego events of the five tokens with amounts in USD
-- (u_approvals, u_transfers), kept around u_cohort, the contracts of the
```

```
-- plan's cohort that >= 3 distinct owners approved on the five tokens in
-- scan A; transfers: value / POW(10, decimals) * price of the UTC day

-- sql/21_usd12m_cohorts.sql: cohort of a freeze (five tokens, >= 3
-- distinct owners inside its score window; the same for t1 with the
-- W0 window), and views of the events of that cohort in the window
CREATE TABLE `contract_rep.u12_cohort_tobs` AS
SELECT spender AS address, COUNT(DISTINCT owner) AS owners_nonzero
FROM `contract_rep.u_approvals`
WHERE block_timestamp >= TIMESTAMP('2025-07-01 00:00:00 UTC')
  AND block_timestamp <  TIMESTAMP('2026-07-01 00:00:00 UTC')
  AND value > 0
  AND spender IN (SELECT address FROM `contract_rep.u_cohort`)
GROUP BY spender HAVING COUNT(DISTINCT owner) >= 3;

-- sql/14_usd_anchor_pairs.sql: latest allowances and pair weights at an
-- anchor, run on the views of the anchor's cohort
SELECT owner, spender,
       SUM(sig(age_days) * vt(usd, @b_allowance)) AS w_er,  -- EndorseRank
       SUM(LEAST(usd, @cap_allowance))           AS w_raw  -- allowance layer
FROM ( -- latest approval per (token, owner, spender) inside the score
       -- window, positive values only, valued at the anchor's day
  SELECT *, ROW_NUMBER() OVER (PARTITION BY token_address, owner, spender
            ORDER BY block_number DESC, log_index DESC) AS rn
  FROM `contract_rep.u_approvals`
  WHERE block_timestamp >= TIMESTAMP(@start_ts)
    AND block_timestamp <= TIMESTAMP(@anchor_ts))
WHERE rn = 1 AND usd > 0
GROUP BY owner, spender;
-- transfer pairs (same window): SUM(sig(age) * vt(usd, @b_transfer))
--   AS w_awp, SUM(usd * sig(age)) AS w_raw, MAX(sig(age)) AS max_sig
```

The scan dates and the extraction times of the July–September logs are recorded in the pipeline's extraction manifests. The extraction script of scan B started at 16:49:33 UTC on 9 October 2026, one second after the registration commit, and created the scan B query at 16:49:40, eight seconds after it.